

Logical Expressions

logical and comparison operators can be mixed to form more complicated logical expressions

an order of operators / precedence analogous to arithmetic operators is used to evaluate such expressions, ie

()
!
>, >=, <, <=
==, !=
&&
||
=

Arithmetic operators have higher precedence, except for ! which is the same level as unary minus -, and assignment which is lower than all operators

note: you can always use () if you are unsure, eg
if ((x < 0.0) || (i <= k)) cout << "\ntrue";
-- () is not required but it's perhaps more readable

assume i = 3, k = 1

Examples:

```
// 1 2 4 3
if ( i+k > 0 || i < k ) cout << "\ntrue";    true
```

true false

```
if ( k > 0 || !(i < k) ) cout << "\ntrue";
```

// 3 4 2 1 -> true || true -> true

```
if ( i > 0 && i < k || i < 7 ) cout << "\ntrue";
```

1 4 2 5 3

```
if ( ( (i > 0) && (i < k) ) || (i < 7) ) cout << "\ntrue";
```

true && false || true

false || true

true

bool and logical variables (see variable types lecture) can also be used in logical expressions, eg

```
int i1 = 1, i2 = 0, i3; bool b1 = true, b2 = false, b3;
```

```
if ( i1 ) cout << "\ni1 is true";
```

```
if ( b2 ) cout << "\nb2 is true";
```

```
if ( i1 && i2 ) cout << "\ni1 is true and i2 is true";
```

recall: a logical variable is an int variable used to represent a binary (true / false) condition -- 0 is false and non-zero is true

logical expressions evaluate to true (1) or false (0) , eg

```
i3 = true || false = true (1)
```

```
i3 = i1 || i2; cout << "\ni3 = " << i3;
```

```
b3 = b1 && b2; cout << "\nb3 = " << b3;
```

```
b3 = true && false = false (0)
```